Stochastic Batched Dynamic Programming

A scaling strategy for joint hardware-policy co-design beyond the exact-DP horizon ceiling

Abstract

Exact Bellman dynamic programming for partially observable Markov decision processes (POMDPs) admits an efficient sufficient-statistic memoization for many problems of interest, but the memoization grows fast enough in horizon K that exact solutions are typically capped at $K \approx 4$–5. Joint hardware-policy co-design, where a continuous hardware parameter $\boldsymbol{c}$ is optimized jointly with the inner Bellman-optimal policy $\pi^\star(\boldsymbol{c})$, inherits this ceiling. We develop a scaling extension: *stochastic batched dynamic programming*, which combines (i) a batched receding-horizon decomposition of the inner Bellman problem into n tractable batches of length K_{batch} and (ii) stochastic Monte-Carlo sampling of batch-boundary trajectories at outer-gradient time, replacing the intractable enumeration over reachable batch-boundary beliefs. The resulting estimator is unbiased, per-sample gradients are exact via the sharp-max envelope theorem, and the wall-clock cost scales linearly in the sample size and the number of batches rather than in the reachable belief count. The construction generalizes naturally and preserves the exactness guarantees of joint dynamic programming on the sampled sub-problems.

Contents

1 Setup and notation **2**

2 The exact-DP horizon ceiling **2**

3 Batched dynamic programming **3**
- 3.1 The value of a batched policy . . . 3
- 3.2 The curse of enumeration over batch-boundary beliefs . . . 3

4 Stochastic batched DP **3**
- 4.1 The estimator . . . 4
- 4.2 Per-sample gradients via the envelope theorem . . . 4
- 4.3 Outer SGD update . . . 4

5 Gradient computation: a detailed derivation **5**
- 5.1 Setup for the two-batch case . . . 5
- 5.2 Identifying every $\boldsymbol{c}$-dependence . . . 5
- 5.3 The REINFORCE / pathwise decomposition . . . 6
- 5.4 Pathwise gradient $\nabla_{\boldsymbol{c}} F(\tau; \boldsymbol{c})$ . . . 6
- 5.5 Score-function gradient $\nabla_{\boldsymbol{c}} \log P(\tau; \boldsymbol{c})$ . . . 7
- 5.6 The full per-trajectory gradient estimator . . . 7
- 5.7 Variance reduction with baselines . . . 8
- 5.8 Generalization to $n > 2$ batches . . . 8
  - 5.8.1 Trajectory factorization across batches . . . 8
  - 5.8.2 REINFORCE applied to the full trajectory . . . 9
  - 5.8.3 Pathwise gradient across all K_{total} steps . . . 9

5.8.4 Per-batch decomposition of the score-function term 9
5.8.5 Equivalent per-batch score-function form (for variance reduction) 10
5.8.6 Variance reduction with per-batch advantage baselines 10
5.8.7 Per-trajectory gradient estimator for general n 11
5.8.8 Sanity check: $n = 2$ reduces to §5.6 11
5.9 Connection back to single-batch joint-DP 12

6 The general algorithm **13**

7 Convergence and bias **13**

8 Connections to existing methods **14**

9 Practical guidance **14**

10 Discussion **15**

1 Setup and notation

A POMDP is a tuple $(\mathcal{X}, \mathcal{S}, \mathcal{Y}, p_0, p, T)$ with hidden state $\boldsymbol{x} \in \mathcal{X}$, action $\boldsymbol{s} \in \mathcal{S}$, observation $\boldsymbol{y} \in \mathcal{Y}$, prior p_0, observation likelihood $p(\boldsymbol{y} \mid \boldsymbol{x}, \boldsymbol{s}; \boldsymbol{c})$, and Bayesian belief update operator $T(\boldsymbol{b}, \boldsymbol{s}, \boldsymbol{y}; \boldsymbol{c})$. The hardware parameter $\boldsymbol{c} \in \mathcal{C}$ parametrises the likelihood and the belief update. Belief at step k is $\boldsymbol{b}_k \in \Delta(\mathcal{X})$, the simplex of distributions on the state space.

A pointwise reward $\Phi(\boldsymbol{x}, \boldsymbol{s}_{1:K}; \boldsymbol{c})$ depends on the realized trajectory and the hardware. For terminal rewards expressible as a functional of the terminal belief $\boldsymbol{b}_K$, write $\Phi_0(\boldsymbol{b}_K; \boldsymbol{c}) = \mathbb{E}_{\boldsymbol{x}|\boldsymbol{b}_K}[\Phi(\boldsymbol{x}, \boldsymbol{s}_{1:K}; \boldsymbol{c})]$. The Bellman recursion at fixed $\boldsymbol{c}$ is

$$V_K(\boldsymbol{b}; \boldsymbol{c}) = \Phi_0(\boldsymbol{b}; \boldsymbol{c}), \tag{1}$$

$$V_k(\boldsymbol{b}; \boldsymbol{c}) = \max_{\boldsymbol{s} \in \mathcal{S}} \mathbb{E}_{\boldsymbol{y}|\boldsymbol{b},\boldsymbol{s},\boldsymbol{c}}\big[V_{k+1}(T(\boldsymbol{b}, \boldsymbol{s}, \boldsymbol{y}; \boldsymbol{c}); \boldsymbol{c})\big], \tag{2}$$

for $k = K-1, \ldots, 0$. The optimal adaptive policy is $\pi_k^\star(\boldsymbol{b}; \boldsymbol{c}) = \arg\max_{\boldsymbol{s}} \mathbb{E}_{\boldsymbol{y}}[V_{k+1}(T(\boldsymbol{b}, \boldsymbol{s}, \boldsymbol{y}; \boldsymbol{c}); \boldsymbol{c})]$. The joint hardware-policy optimum solves

$$\boldsymbol{c}^\star = \arg\max_{\boldsymbol{c} \in \mathcal{C}} V^\star(\boldsymbol{c}), \qquad V^\star(\boldsymbol{c}) := V_0(p_0; \boldsymbol{c}). \tag{3}$$

The outer gradient $dV^\star/d\boldsymbol{c}$ is computed by the sharp-max envelope theorem: at the optimal inner policy, the indirect contribution of policy sensitivity to $\partial V^\star/\partial \boldsymbol{c}$ vanishes, and reverse-mode automatic differentiation through an argmax-frozen Bellman recursion returns an exact, unbiased outer gradient.

2 The exact-DP horizon ceiling

Exact Bellman DP for the recursion of Equations (1)–(2) admits efficient memoization when a low-dimensional sufficient statistic for the belief is available. Two common settings:

Discrete state. The belief is a probability vector on $|\mathcal{X}|$ states. Reachable beliefs from p_0 form a tree of size at most $(|\mathcal{S}| \cdot |\mathcal{Y}|)^K$. Memoization stores one value per reachable belief; the table size is bounded but grows exponentially with K.

Continuous state with sufficient-statistic reduction. When the per-shot likelihood depends on the history only through a finite tuple of integers (e.g., a count-tuple of observations partitioned by action type), the belief at step k is encoded by that tuple, and the reachable-tuple set is the memo's natural index. The number of reachable tuples grows multiplicatively with K, polynomially with the action and observation alphabet sizes. In favourable cases the memo is $\sim 10^7$ at $K = 4$; unfavourable cases (multimodal posteriors, dense action grids) push this to 10^8–10^9 or more by $K = 5$ or 6. Memory becomes the binding constraint.

For practical hardware (modest workstation, ~ 1 TB RAM) the exact-DP ceiling is typically $K_{\max} \in \{4, 5\}$. Beyond that we descend to approximations.

3 Batched dynamic programming

Suppose the deployment horizon K_{total} is much larger than the exact-DP ceiling $K_{\text{batch}} \leq K_{\max}$, with $K_{\text{total}} = n \cdot K_{\text{batch}}$ for some integer n. Define a *batched policy* π^{batch} as follows:

1. At step $k = 0$, with belief $\boldsymbol{b}_0 = p_0$, solve exact Bellman for K_{batch} epochs starting at $\boldsymbol{b}_0$. Use the resulting policy $\pi^{(1)}$ for steps $1, \ldots, K_{\text{batch}}$.
2. At step $k = K_{\text{batch}}$, with belief $\boldsymbol{b}_{K_{\text{batch}}}$ resulting from executing $\pi^{(1)}$ on the realized observations, solve exact Bellman for K_{batch} epochs starting at $\boldsymbol{b}_{K_{\text{batch}}}$. Use $\pi^{(2)}$ for the next K_{batch} steps.
3. Repeat for n batches total.

Each batch is locally Bellman-optimal over its K_{batch}-step subproblem; the overall procedure is a receding-horizon approximation to the full K_{total} Bellman, suboptimal at the start (because $\pi^{(1)}$ plans only K_{batch} steps ahead even though $K_{\text{total}} - K_{\text{batch}}$ steps remain) and tight at the end.

3.1 The value of a batched policy

The expected reward under the batched policy starting from $\boldsymbol{b}_0 = p_0$ is

$$V^{\text{batch}}(p_0;\, \boldsymbol{c}) \;=\; \mathbb{E}_{\boldsymbol{b}_{K_{\text{batch}}} \sim P(\,\cdot\,|p_0, \pi^{(1)},\, \boldsymbol{c})}\big[\, V^{\text{batch}}(\boldsymbol{b}_{K_{\text{batch}}};\, \boldsymbol{c}, \text{remaining } n-1 \text{ batches})\,\big], \tag{4}$$

unfolding to

$$V^{\text{batch}}(p_0;\, \boldsymbol{c}) \;=\; \sum_{\boldsymbol{b}_{K_{\text{batch}}}} P(\boldsymbol{b}_{K_{\text{batch}}} \mid p_0, \pi^{(1)},\, \boldsymbol{c})\, V_0^{\text{batch}_2}(\boldsymbol{b}_{K_{\text{batch}}};\, \boldsymbol{c}), \tag{5}$$

in the two-batch case ($n = 2$), where $V_0^{\text{batch}_2}$ denotes the standard K_{batch} Bellman value at the new starting belief $\boldsymbol{b}_{K_{\text{batch}}}$ over the second batch. The sum is over reachable batch-boundary beliefs.

3.2 The curse of enumeration over batch-boundary beliefs

To compute Equation (5) exactly we would need $V_0^{\text{batch}_2}(\boldsymbol{b}_{K_{\text{batch}}};\, \boldsymbol{c})$ at every reachable $\boldsymbol{b}_{K_{\text{batch}}}$. In the count-tuple-keyed setting this is one fresh K_{batch} Bellman solve per reachable count-tuple after batch 1. At $K_{\text{batch}} = 4$ the reachable count-tuple set is $\sim 10^7$ in unfavourable regimes, and each fresh solve costs ~ 10 seconds: enumeration cost is 10^8 seconds (thousands of CPU-years).

This is the same memo blow-up that capped exact $K_{\text{total}} = K_{\text{batch}} + 1$ in the first place, just relocated to the batch boundary. Batched DP, naively implemented, does not break the exact-DP ceiling.

4 Stochastic batched DP

The standard remedy for an intractable expectation over a large index set is to replace exhaustive enumeration with Monte-Carlo sampling under the natural distribution $P(\boldsymbol{b}_{K_{\text{batch}}} \mid p_0, \pi^{(1)},\, \boldsymbol{c})$.

4.1 The estimator

Sample M trajectories under the current batched policy. For each trajectory m define $\boldsymbol{b}^{(m)}_{K_{\text{batch}}}$ as the realized batch-1 terminal belief. The estimator

$$\widehat{V}^{\text{batch}}(p_0;\, \boldsymbol{c}; \mathcal{M}) \;:=\; \frac{1}{M}\sum_{m=1}^{M} V_0^{\text{batch}_2}\big(\boldsymbol{b}^{(m)}_{K_{\text{batch}}};\, \boldsymbol{c}\big) \tag{6}$$

is an unbiased estimator of Equation (5): the trajectories $\mathcal{M} = \{\boldsymbol{b}^{(m)}_{K_{\text{batch}}}\}_{m=1}^{M}$ are i.i.d. samples from the correct policy-induced distribution, and the integrand is computed exactly per sample by solving fresh K_{batch} Bellman from $\boldsymbol{b}^{(m)}_{K_{\text{batch}}}$.

The variance of the estimator scales as $\text{Var}[V_0^{\text{batch}_2}]/M$ where the variance is over the trajectory distribution; standard concentration inequalities give the $1/\sqrt{M}$ confidence intervals.

4.2 Per-sample gradients via the envelope theorem

For outer optimization we need not just the value but its gradient with respect to $\boldsymbol{c}$. For each sampled trajectory we apply the sharp-max envelope theorem to the K_{batch}-step subproblem starting at $\boldsymbol{b}^{(m)}_{K_{\text{batch}}}$:

1. Run forward K_{batch} Bellman from $\boldsymbol{b}^{(m)}_{K_{\text{batch}}}$. Record the optimal argmax action at every reachable belief in the second batch's subtree.
2. Re-evaluate $V_0^{\text{batch}_2}(\boldsymbol{b}^{(m)}_{K_{\text{batch}}};\, \boldsymbol{c})$ by substituting the recorded argmaxes in place of the max operator. This produces an argmax-free expression in $\boldsymbol{c}$.
3. Apply reverse-mode automatic differentiation to the argmax-free expression.

The envelope theorem certifies that the resulting gradient is exact:

$$\frac{\partial V_0^{\text{batch}_2}(\boldsymbol{b}^{(m)}_{K_{\text{batch}}};\, \boldsymbol{c})}{\partial \boldsymbol{c}} \;=\; \frac{\partial}{\partial \boldsymbol{c}}\bigg|_{\pi^{\star}_{batch_2}\ \text{frozen}} V_0^{\text{batch}_2}(\boldsymbol{b}^{(m)}_{K_{\text{batch}}};\, \boldsymbol{c}). \tag{7}$$

The first batch's contribution to the gradient (through how $\pi^{(1)}$ shapes the sampling distribution of $\boldsymbol{b}_{K_{\text{batch}}}$) follows the same envelope-theorem-frozen recursion applied to batch 1, and contributes a score-function term in $\partial \log P/\partial \boldsymbol{c}$ that can be carried via reverse-mode AD on the trajectory's likelihood, exactly as in the single-batch joint-DP gradient.

The aggregate gradient

$$\widehat{\nabla}_{\boldsymbol{c}} V^{\text{batch}}(p_0;\, \boldsymbol{c}; \mathcal{M}) \;=\; \frac{1}{M}\sum_{m=1}^{M} \frac{\partial V^{(m)}(\boldsymbol{c})}{\partial \boldsymbol{c}} \tag{8}$$

is unbiased for $\nabla_{\boldsymbol{c}} V^{\text{batch}}(p_0;\, \boldsymbol{c})$ and has the standard SGD variance scaling.

4.3 Outer SGD update

The outer optimization becomes Adam (or any preferred stochastic optimizer) on $\boldsymbol{c}$ with mini-batches of size M:

$$\boldsymbol{c}^{(t+1)} = \text{Adam}\big(\boldsymbol{c}^{(t)},\ \widehat{\nabla}_{\boldsymbol{c}} V^{\text{batch}}(p_0;\, \boldsymbol{c}^{(t)}; \mathcal{M}^{(t)}),\ \eta\big). \tag{9}$$

A fresh trajectory mini-batch $\mathcal{M}^{(t)}$ is sampled at each iteration t, so the variance does not compound across iterations. Standard SGD convergence theory applies: under bounded gradient variance and standard step-size schedules the iterates converge in expectation to a stationary point of $V^{\text{batch}}(p_0;\, \boldsymbol{c})$.

5 Gradient computation: a detailed derivation

This section derives the SBDP outer-hardware gradient explicitly. We work the two-batch case ($n = 2$) in full detail, identifying every $\boldsymbol{c}$-dependence and unfolding each chain rule. We then generalize to $n > 2$ batches recursively. No step is skipped; the goal is that the formula in the algorithm box can be derived directly from first principles by following this section.

5.1 Setup for the two-batch case

For $n = 2$ with $K_{\text{total}} = 2K_{\text{batch}}$, the batched value is

$$V^{\text{batch}}(p_0;\, \boldsymbol{c}) \;=\; \mathbb{E}_{\boldsymbol{x}\sim p_0}\, \mathbb{E}_{(\boldsymbol{y}_1,\ldots,\boldsymbol{y}_{K_{\text{batch}}})|\boldsymbol{x},\pi^{[1]}(\boldsymbol{c}),\boldsymbol{c}} \Big[\, V^{[2]}\big(\boldsymbol{b}_{K_{\text{batch}}}(\boldsymbol{x}, \boldsymbol{y}_{1:K_{\text{batch}}}; \boldsymbol{c});\, \boldsymbol{c}\big)\,\Big], \tag{10}$$

where

- $\pi^{[1]}(\boldsymbol{c})$ is the batch-1 policy obtained by exact K_{batch} Bellman from p_0 at parameter $\boldsymbol{c}$. Each action in batch 1 is the local argmax $\boldsymbol{s}_k = \pi^{[1]}(\boldsymbol{b}_{k-1};\, \boldsymbol{c})$.
- $\boldsymbol{b}_k = T(\boldsymbol{b}_{k-1}, \boldsymbol{s}_k, \boldsymbol{y}_k;\, \boldsymbol{c})$ is the Bayesian belief update.
- $V^{[2]}(\boldsymbol{b};\, \boldsymbol{c})$ is the standard K_{batch} Bellman value at starting belief $\boldsymbol{b}$ with parameter $\boldsymbol{c}$. Because batch 2 is the last batch, its local terminal coincides with the global terminal Φ_0.

The trajectory is the tuple $\tau := (\boldsymbol{x}, \boldsymbol{y}_1, \ldots, \boldsymbol{y}_{K_{\text{batch}}})$ with joint density

$$P(\tau;\, \boldsymbol{c}) \;=\; p_0(\boldsymbol{x}) \cdot \prod_{k=1}^{K_{\text{batch}}} p\big(\boldsymbol{y}_k \mid \boldsymbol{x}, \boldsymbol{s}_k(\tau_{<k}, \boldsymbol{c});\, \boldsymbol{c}\big), \tag{11}$$

where $\boldsymbol{s}_k(\tau_{<k}, \boldsymbol{c}) = \pi^{[1]}(\boldsymbol{b}_{k-1}(\tau_{<k}, \boldsymbol{c});\, \boldsymbol{c})$ is the deterministic action under the batch-1 policy.

We want $\nabla_{\boldsymbol{c}} V^{\text{batch}}(p_0;\, \boldsymbol{c})$.

5.2 Identifying every $\boldsymbol{c}$-dependence

Equation (10) has $\boldsymbol{c}$ entering through five distinct paths. We list them explicitly because each contributes to the gradient differently:

(P1) *Batch-1 policy actions* $\boldsymbol{s}_k(\tau_{<k}, \boldsymbol{c})$ via the argmax of the local K_{batch} Bellman.

(P2) *Within-batch-1 observation likelihood* $p(\boldsymbol{y}_k \mid \boldsymbol{x}, \boldsymbol{s}_k;\, \boldsymbol{c})$.

(P3) *Within-batch-1 belief update* $T(\boldsymbol{b}_{k-1}, \boldsymbol{s}_k, \boldsymbol{y}_k;\, \boldsymbol{c})$, which shapes how $\boldsymbol{b}_{K_{\text{batch}}}$ depends on the trajectory.

(P4) *Batch-2 Bellman value at fixed starting belief* $V^{[2]}(\cdot;\, \boldsymbol{c})$, which depends on $\boldsymbol{c}$ through batch-2's likelihood and belief updates.

(P5) *Starting belief input to batch-2,* $\boldsymbol{b}_{K_{\text{batch}}}(\boldsymbol{x}, \boldsymbol{y}_{1:K_{\text{batch}}}; \boldsymbol{c})$. Evaluating $V^{[2]}$ at a starting belief that itself depends on $\boldsymbol{c}$ pulls a chain of (P3) sensitivities in.

We now show that (P1) contributes zero to the gradient under hard argmax, while (P2)–(P5) contribute through two channels: a pathwise term (carrying P3, P4, P5) and a score-function term (carrying P2).

Hard-argmax policy contributes zero. The batch-1 policy is $\boldsymbol{s}_k(\boldsymbol{b}_{k-1}; \boldsymbol{c}) = \arg\max_{\boldsymbol{s}} Q_k^{[1]}(\boldsymbol{b}_{k-1}, \boldsymbol{s};\, \boldsymbol{c})$. For an interior strict maximizer, the argmax is locally constant in $\boldsymbol{c}$: $\partial \boldsymbol{s}_k / \partial \boldsymbol{c} = 0$ almost everywhere in $\boldsymbol{c}$. Hence path (P1) contributes a measure-zero set of jump points, which integrate to zero against $\boldsymbol{c}$-perturbations. Throughout, we therefore treat the realized actions $\boldsymbol{s}^{(m)}_{1:K_{\text{batch}}}$ on each sampled trajectory as $\boldsymbol{c}$-frozen at their recorded values. This is the same locally-constant-argmax assumption used by the sharp-max envelope theorem of the joint-DP framework, applied here to the policy choices.

5.3 The REINFORCE / pathwise decomposition

Let $F(\tau;\, \boldsymbol{c}) := V^{[2]}(\boldsymbol{b}_{K_{\text{batch}}}(\tau, \boldsymbol{c});\, \boldsymbol{c})$. Then $V^{\text{batch}}(p_0;\, \boldsymbol{c}) = \mathbb{E}_{\tau\sim P(\cdot;\, \boldsymbol{c})}[F(\tau;\, \boldsymbol{c})]$. By the log-derivative identity ($\nabla_{\boldsymbol{c}} P = P \cdot \nabla_{\boldsymbol{c}} \log P$),

$$\nabla_{\boldsymbol{c}}\, \mathbb{E}_{\tau\sim P(\cdot;\, \boldsymbol{c})}\big[F(\tau;\, \boldsymbol{c})\big] \;=\; \mathbb{E}_{\tau\sim P(\cdot;\, \boldsymbol{c})}\big[\, \nabla_{\boldsymbol{c}} F(\tau;\, \boldsymbol{c})\,\big] \;+\; \mathbb{E}_{\tau\sim P(\cdot;\, \boldsymbol{c})}\big[\, F(\tau;\, \boldsymbol{c})\, \nabla_{\boldsymbol{c}} \log P(\tau;\, \boldsymbol{c})\,\big]. \tag{12}$$

The first term is the *pathwise gradient* (deterministic chain rule on the integrand at fixed sampled τ). The second term is the *score-function gradient* (REINFORCE-style sensitivity of the trajectory distribution). Both are needed for an unbiased estimator.

5.4 Pathwise gradient $\nabla_{\boldsymbol{c}} F(\tau;\, \boldsymbol{c})$

By chain rule on $F = V^{[2]}\big(\boldsymbol{b}_{K_{\text{batch}}}(\tau, \boldsymbol{c});\, \boldsymbol{c}\big)$,

$$\nabla_{\boldsymbol{c}} F(\tau;\, \boldsymbol{c}) \;=\; \underbrace{\Big[\, \nabla_{\boldsymbol{b}} V^{[2]}\big(\boldsymbol{b}_{K_{\text{batch}}};\, \boldsymbol{c}\big)\,\Big]^{\top} \cdot \nabla_{\boldsymbol{c}} \boldsymbol{b}_{K_{\text{batch}}}(\tau, \boldsymbol{c})}_{\text{(A): input-belief sensitivity}} \;+\; \underbrace{\left.\frac{\partial V^{[2]}\big(\boldsymbol{b};\, \boldsymbol{c}\big)}{\partial \boldsymbol{c}}\right|_{\boldsymbol{b}\text{ held fixed at }\boldsymbol{b}_{K_{\text{batch}}}}}_{\text{(B): standard joint-DP gradient}}. \tag{13}$$

Each term has a concrete computation; we unfold them in turn.

Term (B): standard joint-DP gradient at fixed starting belief. This is the existing joint-DP gradient of the paper's main framework, applied to $V^{[2]}$. Specifically, holding the starting belief fixed at $\boldsymbol{b}^{(m)}_{K_{\text{batch}}}$ on the sampled trajectory, we run forward the K_{batch} Bellman recursion at parameter $\boldsymbol{c}$ to obtain the policy memo for batch 2. Then we apply the sharp-max envelope theorem: freeze the recorded action argmaxes throughout the batch-2 subtree, and compute reverse-mode automatic differentiation on the resulting argmax-free expression for $V^{[2]}(\boldsymbol{b}^{(m)}_{K_{\text{batch}}};\, \boldsymbol{c})$. The result is exact for the batch-2 subproblem.

In the count-tuple-keyed setting, term (B) is computable in time $O(N_{\text{batch}-2-\text{memo}} \cdot \text{cost}(p, T))$, comparable to one K_{batch} Bellman solve.

Term (A): input-belief sensitivity through belief-update chain. This is two distinct factors:

- $\nabla_{\boldsymbol{b}} V^{[2]}(\boldsymbol{b};\, \boldsymbol{c})$: the gradient of the batch-2 Bellman value with respect to the *input belief* $\boldsymbol{b}$, evaluated at $\boldsymbol{b} = \boldsymbol{b}^{(m)}_{K_{\text{batch}}}$.
- $\nabla_{\boldsymbol{c}} \boldsymbol{b}_{K_{\text{batch}}}(\tau, \boldsymbol{c})$: the sensitivity of the batch-1 terminal belief to $\boldsymbol{c}$ along the realized batch-1 trajectory.

Computing $\nabla_{\boldsymbol{b}} V^{[2]}$. The batch-2 Bellman recursion at starting belief $\boldsymbol{b}$ contains $\boldsymbol{b}$ in two places: (i) directly, as the argument of the value function; (ii) through the marginal observation probability $\Pr(\boldsymbol{y} \mid \boldsymbol{b}, \boldsymbol{s};\, \boldsymbol{c}) = \sum_{\boldsymbol{x}} \boldsymbol{b}(\boldsymbol{x})\, p(\boldsymbol{y} \mid \boldsymbol{x}, \boldsymbol{s};\, \boldsymbol{c})$, which appears in every Bellman backup weighting future values. In the count-tuple-keyed framework with $\boldsymbol{b}$ on a K_Φ-grid, $\boldsymbol{b}$ is a K_Φ-dimensional vector; $\nabla_{\boldsymbol{b}} V^{[2]}$ is a K_Φ-vector. It is computable by reverse-mode AD through the argmax-frozen Bellman recursion, treating $\boldsymbol{b}$ as the differentiation target instead of $\boldsymbol{c}$.

Computing $\nabla_{\boldsymbol{c}} \boldsymbol{b}_{K_{\text{batch}}}$. The belief at the end of batch 1 is the recursion

$$\boldsymbol{b}_k = T(\boldsymbol{b}_{k-1}, \boldsymbol{s}_k, \boldsymbol{y}_k;\, \boldsymbol{c}), \qquad k = 1, \ldots, K_{\text{batch}},$$

with $\boldsymbol{s}_k$ and $\boldsymbol{y}_k$ frozen at their recorded values. Differentiating w.r.t. $\boldsymbol{c}$ with frozen $(\boldsymbol{s}_k, \boldsymbol{y}_k)$:

$$\frac{\partial \boldsymbol{b}_k}{\partial \boldsymbol{c}} \;=\; \left.\frac{\partial T}{\partial \boldsymbol{c}}\right|_{\boldsymbol{b}_{k-1}, \boldsymbol{s}_k, \boldsymbol{y}_k} \;+\; \left.\frac{\partial T}{\partial \boldsymbol{b}}\right|_{\boldsymbol{b}_{k-1}, \boldsymbol{s}_k, \boldsymbol{y}_k} \cdot \frac{\partial \boldsymbol{b}_{k-1}}{\partial \boldsymbol{c}}, \tag{14}$$

unfolding from $\partial \boldsymbol{b}_0/\partial \boldsymbol{c} = 0$ (the prior is $\boldsymbol{c}$-independent). This is a standard chain-rule recursion through the belief-update operator and is computable by one reverse-mode AD pass over the K_{batch}-step chain.

Combining (A1) and (A2). Term (A) of Equation (13) is

$$\sum_{i=1}^{K_\Phi} \frac{\partial V^{[2]}}{\partial \boldsymbol{b}_{K_{\text{batch}},i}}\big(\boldsymbol{b}_{K_{\text{batch}}}; \boldsymbol{c}\big) \cdot \frac{\partial \boldsymbol{b}_{K_{\text{batch}},i}}{\partial \boldsymbol{c}}.$$

Both factors are vectors in $\mathbb{R}^{K_\Phi}$; the inner product collapses them into a scalar gradient w.r.t. $\boldsymbol{c}$. In practice the two reverse-mode AD passes are combined into one, treating the entire pipeline (batch-1 belief updates feeding batch-2 Bellman value) as a single computational graph with $\boldsymbol{c}$ as the differentiation target.

5.5 Score-function gradient $\nabla_{\boldsymbol{c}} \log P(\tau;\, \boldsymbol{c})$

From Equation (11),

$$\log P(\tau;\, \boldsymbol{c}) \;=\; \log p_0(\boldsymbol{x}) \;+\; \sum_{k=1}^{K_{\text{batch}}} \log p\big(\boldsymbol{y}_k \mid \boldsymbol{x}, \boldsymbol{s}_k;\, \boldsymbol{c}\big).$$

The prior $p_0(\boldsymbol{x})$ is $\boldsymbol{c}$-independent. Differentiating:

$$\nabla_{\boldsymbol{c}} \log P(\tau;\, \boldsymbol{c}) \;=\; \sum_{k=1}^{K_{\text{batch}}} \nabla_{\boldsymbol{c}} \log p\big(\boldsymbol{y}_k \mid \boldsymbol{x}, \boldsymbol{s}_k;\, \boldsymbol{c}\big), \tag{15}$$

with $\boldsymbol{s}_k$ frozen by the hard-argmax property of §5.2. Each summand is the per-shot likelihood score, computable by reverse-mode AD on the closed-form likelihood $p(\boldsymbol{y} \mid \boldsymbol{x}, \boldsymbol{s};\, \boldsymbol{c})$. Note that this is a vector in $\mathbb{R}^{d_c}$ (one component per hardware coordinate); the score-function gradient is a sum of such vectors across batch-1 shots.

5.6 The full per-trajectory gradient estimator

Combining (12), (13), and (15), the per-trajectory gradient estimator is

$$g^{(m)} \;=\; \underbrace{\Big[\nabla_{\boldsymbol{b}} V^{[2]}\big|_{\boldsymbol{b}^{(m)}_{K_{\text{batch}}}}\Big]^{\top} \cdot \nabla_{\boldsymbol{c}} \boldsymbol{b}^{(m)}_{K_{\text{batch}}}}_{\text{Term A: belief-input} \,\times\, \text{belief chain}} \;+\; \underbrace{\frac{\partial V^{[2]}\big(\boldsymbol{b};\, \boldsymbol{c}\big)}{\partial \boldsymbol{c}}\bigg|_{\boldsymbol{b}=\boldsymbol{b}^{(m)}_{K_{\text{batch}}}}}_{\text{Term B: joint-DP gradient at fixed input}} \;+\; \underbrace{V^{[2]}\big(\boldsymbol{b}^{(m)}_{K_{\text{batch}}};\, \boldsymbol{c}\big) \cdot \sum_{k=1}^{K_{\text{batch}}} \nabla_{\boldsymbol{c}} \log p\big(\boldsymbol{y}_k^{(}}_{\text{Term C: score-function cor}} \tag{16}$$

Aggregating across M samples,

$$\widehat{\nabla}_{\boldsymbol{c}}\, V^{\text{batch}}(p_0;\, \boldsymbol{c}; \mathcal{M}) \;=\; \frac{1}{M} \sum_{m=1}^{M} g^{(m)}. \tag{17}$$

This estimator is unbiased for $\nabla_{\boldsymbol{c}}\, V^{\text{batch}}(p_0;\, \boldsymbol{c})$ by direct substitution of (12) and the law of large numbers.

Roles of the three terms. Term B is what one would write down naively if the trajectory were $\boldsymbol{c}$-independent: the standard joint-DP gradient evaluated at the sampled batch-2 starting belief. Term A corrects for the fact that $\boldsymbol{b}_{K_{\text{batch}}}$ itself depends on $\boldsymbol{c}$ through the chain of belief updates that built it. Term C corrects for the fact that the trajectory's sampling distribution depends on $\boldsymbol{c}$ through the likelihood; without it the estimator is biased toward the trajectories that current $\boldsymbol{c}$ makes likely.

Failure modes if any term is dropped. Dropping Term A produces a bias proportional to the rate at which $\boldsymbol{b}_{K_{\text{batch}}}$ shifts under $\boldsymbol{c}$, scaled by the slope of $V^{[2]}$ in $\boldsymbol{b}$. Dropping Term C produces a bias proportional to the score-function-times-value covariance. Both are problem-dependent; in cases where the likelihood depends on $\boldsymbol{c}$ only weakly across the prior support, Terms A and C can be small, but they are not zero in general.

5.7 Variance reduction with baselines

The score-function term C in (16) is the dominant variance contributor, because it is a product of two random quantities (the value $V^{[2]}$ and the per-shot scores). Standard variance reduction subtracts a $\boldsymbol{c}$-only baseline $B(\boldsymbol{c})$ from $V^{[2]}$ before multiplying by the score:

$$g^{(m)}_{\text{reduced}} \;=\; (\text{A}) + (\text{B}) + \big[V^{[2]}(\boldsymbol{b}^{(m)}_{K_{\text{batch}}};\, \boldsymbol{c}) - B(\boldsymbol{c})\big] \cdot \sum_k \nabla_{\boldsymbol{c}} \log p(\boldsymbol{y}^{(m)}_k \mid \boldsymbol{x}^{(m)}, \boldsymbol{s}^{(m)}_k; \boldsymbol{c}). \tag{18}$$

The estimator remains unbiased for any $B(\boldsymbol{c})$ that does not depend on $\tau^{(m)}$, because $\mathbb{E}_\tau[\nabla_{\boldsymbol{c}} \log P(\tau;\, \boldsymbol{c})] = 0$ identically.

Choice of baseline. Two standard choices:

- *Empirical mean:* $B(\boldsymbol{c}) = \frac{1}{M}\sum_{m'=1}^{M} V^{[2]}(\boldsymbol{b}^{(m')}_{K_{\text{batch}}};\, \boldsymbol{c})$. Cheap (free, given the values are already computed) but biased if used naively (because B depends on the same samples whose scores it modifies). The standard remedy is leave-one-out: $B^{(m)}(\boldsymbol{c}) = \frac{1}{M-1}\sum_{m' \neq m} V^{[2]}(\boldsymbol{b}^{(m')}_{K_{\text{batch}}};\, \boldsymbol{c})$.
- *Local Bellman value:* $B(\boldsymbol{c}) = V^{[1]}_{\text{local}}(p_0;\, \boldsymbol{c})$, the K_{batch} Bellman value of batch 1 at p_0 with local terminal Φ_0. This is computable for free during the batch-1 Bellman solve and is the optimal baseline in the sense of minimum variance among $\boldsymbol{c}$-only constants when the local terminal of batch 1 matches the global terminal of $V^{[2]}$.

The local-Bellman baseline is the natural "advantage-function" choice and we recommend it when the local and global terminals coincide (e.g., $\Phi_0 = -\text{Var}(\boldsymbol{x} \mid \boldsymbol{b})$ applied at end of every batch including the final one).

5.8 Generalization to $n > 2$ batches

We now extend the derivation of §5.1–5.6 to general n batches. The structure mirrors the two-batch case exactly; the additional content is the per-batch decomposition of the score-function term and the corresponding advantage-form variance reduction.

5.8.1 Trajectory factorization across batches

For n batches with $K_{\text{total}} = nK_{\text{batch}}$, denote the per-batch sub-trajectories

$$\tau_j := \big(\boldsymbol{y}_{(j-1)K_{\text{batch}}+1}, \ldots, \boldsymbol{y}_{jK_{\text{batch}}}\big), \qquad j = 1, \ldots, n,$$

and the full trajectory $\tau := (\boldsymbol{x}, \tau_1, \ldots, \tau_n)$. Let $\boldsymbol{b}^{\text{bdy}}_j := \boldsymbol{b}_{jK_{\text{batch}}}$ denote the belief at the end of batch j (so $\boldsymbol{b}^{\text{bdy}}_0 = p_0$ and $\boldsymbol{b}^{\text{bdy}}_n = \boldsymbol{b}_{K_{\text{total}}}$).

The trajectory density factorizes as a single product over all K_{total} shots,

$$P(\tau;\, \boldsymbol{c}) \;=\; p_0(\boldsymbol{x}) \cdot \prod_{k=1}^{K_{\text{total}}} p\big(\boldsymbol{y}_k \mid \boldsymbol{x}, \boldsymbol{s}_k;\, \boldsymbol{c}\big), \tag{19}$$

which by grouping shots into batches becomes

$$P(\tau;\, \boldsymbol{c}) \;=\; p_0(\boldsymbol{x}) \cdot \prod_{j=1}^{n} P_j\big(\tau_j \mid \boldsymbol{x}, \boldsymbol{b}^{\text{bdy}}_{j-1};\, \boldsymbol{c}\big), \quad \text{where} \quad P_j(\tau_j \mid \boldsymbol{x}, \boldsymbol{b}^{\text{bdy}}_{j-1};\, \boldsymbol{c}) = \prod_{k \in \text{batch } j} p(\boldsymbol{y}_k \mid \boldsymbol{x}, \boldsymbol{s}_k;\, \boldsymbol{c}). \tag{20}$$

The per-batch density P_j depends on $\boldsymbol{c}$ through both the within-batch likelihoods and the local Bellman policy that selects $\boldsymbol{s}_k$ for $k \in$ batch j. As before, the policy contribution vanishes under hard argmax, so $\boldsymbol{s}_k$ values are frozen at recorded argmaxes.

5.8.2 REINFORCE applied to the full trajectory

Because the full trajectory factorization (19) contains all K_{total} shots in one product, the REINFORCE identity (12) applies to the full trajectory directly. With $F(\tau;\, \boldsymbol{c}) := \Phi_0(\boldsymbol{b}_{K_{\text{total}}}(\tau, \boldsymbol{c});\, \boldsymbol{c})$,

$$\nabla_{\boldsymbol{c}} V^{\text{batch}}(p_0;\, \boldsymbol{c}) \;=\; \mathbb{E}_{\tau\sim P(\cdot;\,\boldsymbol{c})}\big[\, \nabla_{\boldsymbol{c}} F(\tau;\, \boldsymbol{c})\,\big] \;+\; \mathbb{E}_{\tau\sim P(\cdot;\,\boldsymbol{c})}\big[\, F(\tau;\, \boldsymbol{c}) \cdot \nabla_{\boldsymbol{c}} \log P(\tau;\, \boldsymbol{c})\,\big]. \tag{21}$$

Both terms unfold across all K_{total} steps; the per-batch decomposition arises naturally because the underlying recursion has batch boundaries built into the policy.

5.8.3 Pathwise gradient across all K_{total} steps

The terminal reward $\Phi_0(\boldsymbol{b}_{K_{\text{total}}}(\tau, \boldsymbol{c});\, \boldsymbol{c})$ depends on $\boldsymbol{c}$ through (i) any direct $\boldsymbol{c}$-dependence in the functional form of Φ_0 and (ii) the chain of K_{total} belief updates. By chain rule,

$$\nabla_{\boldsymbol{c}} F(\tau;\, \boldsymbol{c}) \;=\; \left.\frac{\partial \Phi_0(\boldsymbol{b};\, \boldsymbol{c})}{\partial \boldsymbol{c}}\right|_{\boldsymbol{b}=\boldsymbol{b}_{K_{\text{total}}}} \;+\; \Big[\nabla_{\boldsymbol{b}} \Phi_0(\boldsymbol{b}_{K_{\text{total}}};\, \boldsymbol{c})\Big]^{\top} \cdot \nabla_{\boldsymbol{c}}\, \boldsymbol{b}_{K_{\text{total}}}(\tau, \boldsymbol{c}), \tag{22}$$

with the belief-chain derivative

$$\frac{\partial \boldsymbol{b}_k}{\partial \boldsymbol{c}} \;=\; \left.\frac{\partial T}{\partial \boldsymbol{c}}\right|_{\boldsymbol{b}_{k-1},\boldsymbol{s}_k,\boldsymbol{y}_k} \;+\; \left.\frac{\partial T}{\partial \boldsymbol{b}}\right|_{\boldsymbol{b}_{k-1},\boldsymbol{s}_k,\boldsymbol{y}_k} \cdot \frac{\partial \boldsymbol{b}_{k-1}}{\partial \boldsymbol{c}}, \qquad k = 1, \ldots, K_{\text{total}}, \tag{23}$$

unfolding from $\partial \boldsymbol{b}_0 / \partial \boldsymbol{c} = 0$. Equation (23) is the same belief-chain recursion as Equation (14) but extended over all K_{total} steps, not just the first K_{batch}. In implementation it is one reverse-mode AD pass over the full trajectory's belief-update chain. Note that the batch boundaries do not appear specially in this term: the belief update is the same operator at every step, regardless of which batch the step belongs to.

Comparison with the two-batch pathwise term. In the two-batch derivation (13) the pathwise term split into an "input belief" piece (A) and a "fixed-input batch-2 Bellman gradient" piece (B). Here we have folded both pieces into a single chain rule expression (22). The split is equivalent: the "fixed-input batch-2 Bellman gradient" of the two-batch case corresponds to differentiating the second batch's belief-update chain with the second batch's input frozen, while the "input belief sensitivity" corresponds to differentiating the first batch's chain. Both flatten into the same total chain over K_{total} steps.

5.8.4 Per-batch decomposition of the score-function term

The full score function unfolds across all shots:

$$\nabla_{\boldsymbol{c}} \log P(\tau;\, \boldsymbol{c}) \;=\; \sum_{k=1}^{K_{\text{total}}} \nabla_{\boldsymbol{c}} \log p\big(\boldsymbol{y}_k \mid \boldsymbol{x}, \boldsymbol{s}_k;\, \boldsymbol{c}\big) \;=\; \sum_{j=1}^{n} \underbrace{\nabla_{\boldsymbol{c}} \log P_j\big(\tau_j \mid \boldsymbol{x}, \boldsymbol{b}_{j-1}^{\text{bdy}};\, \boldsymbol{c}\big)}_{\sum_{k\in\text{batch } j} \nabla_{\boldsymbol{c}} \log p(\boldsymbol{y}_k|\boldsymbol{x},\boldsymbol{s}_k;\,\boldsymbol{c})}. \tag{24}$$

The grouping into n per-batch contributions is purely notational at this stage — the per-batch sums are just additive. But it becomes load-bearing in the next step, where each per-batch term gets multiplied by the residual remaining-horizon value from that batch onwards rather than by the global terminal.

5.8.5 Equivalent per-batch score-function form (for variance reduction)

The "raw" REINFORCE gradient (21) multiplies the score function by the global terminal $F(\tau;\, \boldsymbol{c}) = \Phi_0(\boldsymbol{b}_{K_{\text{total}}};\, \boldsymbol{c})$. By the same identity that proves $\mathbb{E}_\tau[\nabla_{\boldsymbol{c}} \log P(\tau;\, \boldsymbol{c})] = 0$ for an arbitrary constant baseline, we can substitute any *$\boldsymbol{c}$-only* or *causally-prior* quantity for the score-function multiplier within each batch's term, without introducing bias. Specifically:

Lemma 1 (Per-batch score-function multiplier substitution)**.** *For each $j = 1, \ldots, n$, let $W_j(\boldsymbol{c}, \tau_{<j})$ be any function depending on $\boldsymbol{c}$ and on the trajectory* prior *to batch j (i.e., on $\boldsymbol{x}$ and on $\tau_1, \ldots, \tau_{j-1}$). Then*

$$\mathbb{E}_\tau\big[\, W_j(\boldsymbol{c}, \tau_{<j}) \cdot \nabla_{\boldsymbol{c}} \log P_j(\tau_j \mid \boldsymbol{x}, \boldsymbol{b}_{j-1}^{\text{bdy}};\, \boldsymbol{c})\,\big] \;=\; 0. \tag{25}$$

Proof. By the tower property of conditional expectation,

$$\mathbb{E}_\tau\big[W_j \cdot \nabla_{\boldsymbol{c}} \log P_j\big] \;=\; \mathbb{E}_{\tau_{<j}}\Big[\, W_j(\boldsymbol{c}, \tau_{<j}) \cdot \mathbb{E}_{\tau_j \mid \tau_{<j}}\big[\nabla_{\boldsymbol{c}} \log P_j(\tau_j \mid \boldsymbol{x}, \boldsymbol{b}_{j-1}^{\text{bdy}};\, \boldsymbol{c})\big]\,\Big],$$

and the inner conditional expectation is zero because $\mathbb{E}[\nabla_{\boldsymbol{c}} \log p] = 0$ for any parametric density. □

This means we can replace the score-function multiplier inside batch j's term with the *remaining-horizon value* $V_{\text{global}}^{(j+1)}(\boldsymbol{b}_j^{\text{bdy}};\, \boldsymbol{c})$ plus or minus any $\boldsymbol{c}$-only quantity:

$$\mathbb{E}_\tau\big[F \cdot \nabla_{\boldsymbol{c}} \log P_j\big] \;=\; \mathbb{E}_\tau\big[V_{\text{global}}^{(j+1)}(\boldsymbol{b}_j^{\text{bdy}};\, \boldsymbol{c}) \cdot \nabla_{\boldsymbol{c}} \log P_j\big], \tag{26}$$

because the difference $F - V_{\text{global}}^{(j+1)}(\boldsymbol{b}_j^{\text{bdy}};\, \boldsymbol{c})$ has zero conditional expectation given $\tau_{<j+1}$ (it is exactly the residual stochasticity in batches $j+1, \ldots, n$ given the boundary belief), and that residual is independent of batch j's score function only when integrated against batch j's score (which is what Lemma 1 certifies).

Combining (21), (24), and (26) yields the *per-batch* REINFORCE form:

$$\boxed{\nabla_{\boldsymbol{c}} V^{\text{batch}}(p_0;\, \boldsymbol{c}) \;=\; \mathbb{E}_\tau\Big[\, \nabla_{\boldsymbol{c}} F(\tau;\, \boldsymbol{c}) \;+\; \sum_{j=1}^{n} V_{\text{global}}^{(j+1)}\big(\boldsymbol{b}_j^{\text{bdy}};\, \boldsymbol{c}\big) \cdot \nabla_{\boldsymbol{c}} \log P_j(\tau_j \mid \boldsymbol{x}, \boldsymbol{b}_{j-1}^{\text{bdy}};\, \boldsymbol{c})\,\Big].} \tag{27}$$

Here $V_{\text{global}}^{(n+1)} := \Phi_0(\boldsymbol{b};\, \boldsymbol{c})$ by convention (so the $j = n$ term collapses to the standard score function multiplied by the realized terminal).

5.8.6 Variance reduction with per-batch advantage baselines

By the same lemma, we can subtract any $\boldsymbol{c}$-only or causally-prior quantity from $V_{\text{global}}^{(j+1)}(\boldsymbol{b}_j^{\text{bdy}};\, \boldsymbol{c})$ inside the j-th term without introducing bias. Define the per-batch advantage

$$A_j(\boldsymbol{b}_j^{\text{bdy}};\, \boldsymbol{c}) \;:=\; V_{\text{global}}^{(j+1)}\big(\boldsymbol{b}_j^{\text{bdy}};\, \boldsymbol{c}\big) - V_{\text{global}}^{(j)}\big(\boldsymbol{b}_{j-1}^{\text{bdy}};\, \boldsymbol{c}\big). \tag{28}$$

This is the *realized-minus-expected* value of batch j: it measures how much unexpectedly more (or less) value batch j delivered relative to what was anticipated from $\boldsymbol{b}_{j-1}^{\text{bdy}}$ at the start of the batch. By Lemma 1, $\mathbb{E}[V_{\text{global}}^{(j)}(\boldsymbol{b}_{j-1}^{\text{bdy}};\, \boldsymbol{c}) \cdot \nabla_{\boldsymbol{c}} \log P_j] = 0$, so substituting $V_{\text{global}}^{(j+1)}$ with A_j leaves the gradient unbiased:

$$\nabla_{\boldsymbol{c}} V^{\text{batch}}(p_0;\, \boldsymbol{c}) \;=\; \mathbb{E}_\tau\Big[\, \nabla_{\boldsymbol{c}} F(\tau;\, \boldsymbol{c}) \;+\; \sum_{j=1}^{n} A_j(\boldsymbol{b}_j^{\text{bdy}};\, \boldsymbol{c}) \cdot \nabla_{\boldsymbol{c}} \log P_j(\tau_j \mid \boldsymbol{x}, \boldsymbol{b}_{j-1}^{\text{bdy}};\, \boldsymbol{c})\,\Big]. \tag{29}$$

Equation (29) is the actor-critic / advantage form. Variance is typically much smaller than (27) because A_j has zero conditional mean (by definition), while $V_{\text{global}}^{(j+1)}$ has the full per-trajectory variance.

Estimating $V^{(j)}_{\text{global}}$. The advantage form requires $V^{(j)}_{\text{global}}$ at the boundary beliefs, but we do not have a closed form for these in the general SBDP setting. Three practical estimators:

- *Empirical mean over samples*: $\widehat{V}^{(j)}_{\text{global}}(\boldsymbol{b};\boldsymbol{c}) \approx \frac{1}{M}\sum_{m'=1}^{M} F(\tau^{(m')};\boldsymbol{c})$, treating samples as i.i.d. from the trajectory distribution. Cheap; introduces a small leave-one-out bias unless one excludes the current sample's contribution.
- *Local Bellman value*: $\widehat{V}^{(j)}_{\text{global}}(\boldsymbol{b};\boldsymbol{c}) \approx V^{[j]}_{\text{local}}(\boldsymbol{b};\boldsymbol{c})$, the standard K_{batch} Bellman value with terminal Φ_0 at end of batch j. Free during the local Bellman solve. Optimal as a baseline when local terminal coincides with global terminal.
- *Bootstrap from sampled trajectories*: simulate one further batch under the current policy starting from $\boldsymbol{b}^{\text{bdy}}_j$ and record its terminal value. Equivalent to one-step temporal-difference estimation.

Any of these is unbiased as a baseline (Lemma 1) but they trade off variance vs. implementation cost.

5.8.7 Per-trajectory gradient estimator for general n

Combining (22) and (29), the per-trajectory gradient estimator with advantage variance reduction is

$$g^{(m)}_n = \underbrace{\left[\frac{\partial \Phi_0}{\partial \boldsymbol{c}} + (\nabla_{\boldsymbol{b}}\Phi_0)^\top \cdot \nabla_{\boldsymbol{c}} \boldsymbol{b}^{(m)}_{K_{\text{total}}}\right]}_{\text{pathwise (single chain over } K_{\text{total}})} + \sum_{j=1}^{n} \underbrace{A^{(m)}_j \cdot \sum_{k\in\text{batch } j} \nabla_{\boldsymbol{c}} \log p\big(\boldsymbol{y}^{(m)}_k \mid \boldsymbol{x}^{(m)}, \boldsymbol{s}^{(m)}_k; \boldsymbol{c}\big)}_{\text{per-batch advantage}\times\text{batch-}j\text{ score}}. \tag{30}$$

Aggregating across M samples:

$$\widehat{\nabla}_{\boldsymbol{c}} V^{\text{batch}}(p_0;\boldsymbol{c};\mathcal{M}) = \frac{1}{M}\sum_{m=1}^{M} g^{(m)}_n. \tag{31}$$

This is unbiased for $\nabla_{\boldsymbol{c}} V^{\text{batch}}$ for any choice of advantage baselines $\widehat{V}^{(j)}_{\text{global}}$ that depend only on $\boldsymbol{c}$ and $\tau^{(m)}_{<j}$.

Per-step cost decomposition. Per sample m:

- One reverse-mode AD pass over the full K_{total}-step belief-update chain: pathwise term. Cost: $O(K_{\text{total}} \cdot K_\Phi \cdot d_c)$.
- For each of n batch boundaries: one K_{batch} Bellman solve to obtain $V^{[j]}_{\text{local}}$ for the local-Bellman baseline (or alternative). Cost: $O(n \cdot \text{cost}(K_{\text{batch}} \text{ Bellman}))$.
- One additional reverse-mode AD pass for each of the K_{total} score-function terms: cost $O(K_{\text{total}} \cdot d_c)$.

Total per sample: dominated by $n \cdot \text{cost}(K_{\text{batch}} \text{ Bellman})$ for typical problems. Aggregate over M samples in parallel.

5.8.8 Sanity check: $n = 2$ reduces to §5.6

At $n = 2$, the per-batch sum in (30) contains two terms: $j = 1$ (advantage of batch 1) and $j = 2$ (advantage of batch 2).

- $A_1 = V^{(2)}_{\text{global}}(\boldsymbol{b}^{\text{bdy}}_1;\boldsymbol{c}) - V^{(1)}_{\text{global}}(p_0;\boldsymbol{c})$. The second term is $\boldsymbol{c}$-only and frequently dropped; the first term is exactly the $V^{[2]}(\boldsymbol{b}^{(m)}_{K_{\text{batch}}};\boldsymbol{c})$ that appeared as Term C of Equation (16).
- $A_2 = V^{(3)}_{\text{global}}(\boldsymbol{b}^{\text{bdy}}_2;\boldsymbol{c}) - V^{(2)}_{\text{global}}(\boldsymbol{b}^{\text{bdy}}_1;\boldsymbol{c}) = \Phi_0(\boldsymbol{b}_{K_{\text{total}}};\boldsymbol{c}) - V^{[2]}(\boldsymbol{b}^{\text{bdy}}_1;\boldsymbol{c})$. The first piece is the realized terminal; the second is the predicted-from-batch-1-end value.

The pathwise gradient (22) over the $K_{\text{total}} = 2K_{\text{batch}}$ chain coincides with Term A + Term B of Equation (13). The score-function term multiplied by A_1 corresponds to the score-function-times-baseline-corrected version of Term C. The score-function term multiplied by A_2 contains the batch-2 score-function contribution (which was implicitly handled inside Term B in the two-batch derivation, since Term B used exact within-batch Bellman to integrate over batch-2's outcomes analytically).

In short: the $n = 2$ case of (30) is equivalent to Equation (16) after baseline shifting and after recognizing that the batch-2 score-function contribution is implicitly captured inside the batch-2 exact Bellman gradient. The two derivations agree.

Why one might prefer $n = 2$'s explicit form. In the $n = 2$ derivation we kept the second batch's exact-Bellman-with-summation-over-outcomes structure as Term B; this avoids one of the two stochastic score-function contributions and reduces variance. For general n, the analog is to use exact Bellman within the *last* batch (which by definition has terminal aligned with the global terminal) and stochastic batched DP only for batch boundaries $j = 1, \ldots, n-1$. This hybrid keeps batch n's contribution deterministic and fully Bellman-exact, identical to the existing joint-DP gradient machinery, while only sampling at boundaries $j < n$.

5.9 Connection back to single-batch joint-DP

For $n = 1$ (single batch with $K_{\text{total}} = K_{\text{batch}}$), there is no batch-boundary sampling: the entire integral over τ is performed exactly inside the Bellman recursion via the marginal observation probability sums. Equation (**??**) reduces to

$$\nabla_{\boldsymbol{c}} V^{\text{batch}}(p_0;\, \boldsymbol{c}) \;=\; \nabla_{\boldsymbol{c}} V^{[1]}(p_0;\, \boldsymbol{c}) \;=\; \left.\frac{\partial V^{[1]}(\boldsymbol{b};\, \boldsymbol{c})}{\partial \boldsymbol{c}}\right|_{\boldsymbol{b}=p_0,\pi^\star \text{ frozen}},$$

which is exactly the sharp-max envelope-theorem gradient of the paper's main framework. The score-function term vanishes because there are no batch boundaries and the $\mathbb{E}_\tau$ integration is performed exactly. This recovers the exact-Bellman joint-DP gradient as the $n = 1$ specialization of SBDP. For $n \geq 2$ the additional score-function terms appear precisely because the integration over batch-boundary trajectories is replaced by Monte-Carlo sampling.

6 The general algorithm

Algorithm 1 Stochastic Batched DP

Require: Initial $\boldsymbol{c}^{(0)}$, total horizon K_{total}, batch length K_{batch}, sample size M, learning rate η, iterations T

```
for t = 0, 1, ..., T − 1 do
    g ← 0
    for m = 1, ..., M (parallel) do
        b ← p_0
        Initialize trajectory τ_m ← ()
        for batch = 1, ..., n = K_total/K_batch do
            (V_batch, π_batch, memo_batch) ← Bellman(b, K_batch, c^(t))
            for k = 1, ..., K_batch do
                s ← π_batch(b)
                x ~ b; y ~ p(· | x, s; c^(t))
                b ← T(b, s, y; c^(t))
                Append (s, y, b) to τ_m
            end for
        end for
        V_m ← Φ_0(b; c^(t))                    ▷ terminal reward at trajectory leaf
        g_m ← EnvelopeGrad(V_m, τ_m, memos, c^(t))
        g ← g + g_m
    end for
    g ← g/M
    c^(t+1) ← Adam(c^(t), g, η)
end for
return c^(T)
```

Complexity per outer iteration: $M \times n_{\text{batches}} \times \text{cost}(K_{\text{batch}}\ \text{Bellman})$. In typical regimes, $\text{cost}(K_{\text{batch}}\ \text{Bellman}) \approx 10$ seconds, so a single outer gradient step at $M = 10$ and $n = 4$ takes ~ 400 seconds wall-clock without parallelism, or ~ 40 seconds with M-way parallelism on a single multicore node. A complete training run of $T = 500$ iterations finishes in 5–7 hours on standard hardware.

7 Convergence and bias

Per-sample exactness. Each V_m in Equation (6) is computed by exact Bellman within its K_{batch} subproblem, with the sharp-max envelope-theorem gradient. No smoothing temperature, no policy approximation, no biased inner optimizer. The per-sample value and gradient are both exact for the trajectory's contribution to the estimator.

Estimator bias and variance. The estimator $\widehat{V}^{\text{batch}}(p_0;\, \boldsymbol{c}; \mathcal{M})$ is unbiased for the true batched value (which is itself an approximation of the full K_{total} Bellman value). Its variance scales as σ^2/M where σ^2 depends on the spread of $V_0^{\text{batch}_2}$ across reachable batch-boundary beliefs. In well-conditioned problems σ^2 is small and $M = 10$–30 suffices; in problems with bimodal or multimodal batch-boundary belief distributions M needs to grow accordingly.

Convergence rate. Under standard SGD assumptions (bounded gradient variance, diminishing step sizes), the outer iterates converge to a stationary point of $V^{\text{batch}}(p_0;\, \boldsymbol{c})$ at the standard

$1/\sqrt{T}$ rate. Since the per-sample gradient is exact rather than smoothed, the asymptotic noise floor is set by sampling variance alone, not by gradient bias.

Approximation gap to full K_{total} Bellman. The batched policy is a strict approximation of the full K_{total} Bellman policy: each batch's lookahead is truncated to K_{batch}, missing the tail value of subsequent batches. The gap $V_0(p_0; \boldsymbol{c}) - V^{\text{batch}}(p_0; \boldsymbol{c}) \geq 0$ is regime-dependent. When the batch-boundary posterior is unimodal and well-concentrated (e.g., K_{batch} is large enough to "set up" a well-localized belief for subsequent batches), the gap is small. When the batch-boundary posterior is multimodal, the gap can be large because batched DP fails to recognize that $\pi^{(1)}$ should anticipate the disambiguation work of $\pi^{(2)}$.

8 Connections to existing methods

Stochastic gradient descent. The construction is structurally identical to minibatch SGD on neural networks: full-batch gradient over a finite-but-huge index set (here, reachable batch-boundary beliefs) is replaced by a small stochastic mini-batch. The unbiasedness, variance, and convergence properties carry over directly.

Stochastic Dual Dynamic Programming (SDDP). Pereira and Pinto's SDDP for multi-stage stochastic linear programs uses sampled forward-backward sweeps to build cutting-plane approximations of the value function. Stochastic Batched DP is the POMDP analogue: scenarios are replaced by belief trajectories, cutting-plane approximations are replaced by exact Bellman within each batch. The receding-horizon decomposition is a standard MPC pattern in both communities.

Sequential Bayesian experimental design. Simulation-based sequential OED (Huan and Marzouk; Long, Marzouk, and Wang) uses MC sampling of belief trajectories at outer optimization time, but typically with rung-3 myopic information-gain inner policies rather than exact Bellman. Stochastic Batched DP can be viewed as the upgrade of these methods from "myopic" to "Bellman within each batch".

Differentiable POMDP planning. Existing differentiable-POMDP work (deep RL with NN policies, Karkus et al. QMDP-net, etc.) uses NN-parametrized policies with end-to-end gradient training. These methods exhibit a known pathology in joint hardware-policy optimization: the NN policy can absorb the training signal and leave the hardware near initialization (the "hardware-light fixed point"). Stochastic Batched DP sidesteps this by using exact Bellman within each batch, a non-NN inner solver that does not have a free policy capacity to absorb hardware-task signal.

Sample-based POMDP solvers (POMCP, DESPOT). These methods perform Monte-Carlo tree search over the belief space at policy-inference time. Stochastic Batched DP uses sampling at outer-gradient time instead, which is a different point of insertion: the inner policy is exact Bellman (sample-free), and sampling enters only to estimate the outer-hardware gradient.

9 Practical guidance

Choosing K_{batch}. Pick the largest K_{batch} for which exact Bellman fits in the available memory budget. Going larger reduces the approximation gap to full Bellman; going smaller reduces per-iteration wall-clock at the cost of suboptimal early-batch lookahead.

Choosing M. Start at $M = 10$. Increase if the gradient-step direction is noisy enough to slow down Adam convergence (visible as oscillation in the loss trajectory). $M = 30$–50 is a typical regime.

Re-sampling cadence. Sample fresh trajectories at every outer iteration. Re-using trajectories across multiple SGD steps introduces gradient bias because the sampling distribution depends on $\boldsymbol{c}^{(t)}$.

Parallelism. The M trajectory simulations and Bellman solves are embarrassingly parallel. On a C-core machine, target $M \leq C/k_{\text{Bellman}}$ where k_{Bellman} is the number of cores each individual Bellman solve uses effectively (typically 8–64).

Diagnostics. Track the per-batch values $V_0^{\text{batch}_j}$ and the spread of $V^{(m)}$ across samples. A high spread indicates that batch-boundary beliefs are diverse, which is healthy for sampling but suggests K_{batch} may be too small. A monotonically decreasing batch-boundary belief variance across batches indicates progressive disambiguation, which is the regime where batched DP works well.

10 Discussion

Stochastic Batched DP fills a gap in the relaxation hierarchy between exact Bellman DP (rung 1, $K_{\max}$-limited) and the policy-approximation rungs 2–4 (certainty-equivalent, myopic, Gaussian-trace-EKF). It preserves exactness of the inner solver within each batch and exactness of per-sample gradients via the envelope theorem, while breaking the reachable-belief enumeration via Monte-Carlo sampling. The resulting method is well-suited to problems where (a) exact Bellman is feasible at modest K_{batch}, (b) the deployment horizon is much larger, and (c) the outer hardware optimization is the central computational expense.

The framework is particularly natural when combined with point-based value iteration (PBVI) or its rho-POMDP extension as the inner solver. PBVI's α-vector representation makes the per-sample K_{batch} Bellman value $V_0^{\text{batch}_j}(\boldsymbol{b})$ evaluatable at any belief $\boldsymbol{b}$ in $O(|\Gamma| \cdot |\mathcal{X}|)$ time without re-solving from scratch. Combined with the stochastic-batched outer SGD, this lifts the horizon ceiling from $K_{\max} \approx 5$ (exact memoization) to K_{total} of practically arbitrary size.